



JUNIOR PAY

Smart, Secure Pocket-Money & Digital Allowance Management for Modern Families

TECHNICAL & PRODUCT DOCUMENTATION



Built by Shivam Nanda

B.Tech CSE, Government College of Engineering, Gaya (BEU)
Co-Founder, MindForge
Host, "Learning Now by Shivam Nanda" (YouTube)



Live Application

junior-pay.onrender.com
LinkedIn: linkedin.com/in/shivam-nanda-472289418



Technology Stack

Next.js + PostgreSQL + Prisma + Razorpay
Full-Stack Web Application



Document Details

Compiled: September 2026
Source: Full development chat log (Google Gemini)



₹150 Approved

at Burger King • by Mom



₹400 Requested

at Book Store • by Junior



Smarter
Families
Brighter
Tomorrows



Dad
(Primary)



Mom
(Co-Guardian)



Junior

Table of Contents

1.	Executive Summary	3
2.	Origin Story — From Idea to Product	4
3.	Problem Statement	5
4.	How Junior Pay Works (Core Flow)	6
5.	Feature List (Final Build)	7
6.	Screens & User Journeys	8
7.	Tech Stack & Architecture	9
8.	Database Schema (Prisma Models)	10
9.	Development Timeline & Iterations	11
10.	Security & Engineering Edge Cases	12
11.	Notable Engineering Challenges Solved	13
12.	Market Differentiation vs. Existing Solutions	14
13.	Limitations & Known Constraints	15
14.	Deployment	16
15.	How to Use — Setup & User Guide	17
16.	Future Roadmap	18
17.	About the Author	19

1. Executive Summary

Junior Pay is a full-stack FinTech web application that lets children request payments to a merchant's UPI VPA — scanned from a QR code or typed manually — and routes that request as a real-time approval to a linked parent, who reviews the amount and the child's note before paying. It was designed, built, and shipped end-to-end by Shivam Nanda over a structured, self-taught 15-day build cycle, using Next.js, PostgreSQL, Prisma, and Razorpay (Test Mode), and is deployed live on Render.

The project began as a pitch idea aimed at [Shivam Nanda](#), a senior software engineer at PhonePe, evolved through a competitive-analysis phase against NPCI's UPI Circle feature, and was rebuilt from scratch as a production-style, multi-tenant SaaS platform once the scope and differentiation were clear. The final product is named **Junior Pay**.

Attribute	Detail
Build duration	~15 focused development days (structured curriculum, Phase 1–5)
Core users	Parents / Guardians and Juniors (children), in a multi-tenant, many-to-many relationship
Primary stack	Next.js 16 (App Router) · TypeScript · Tailwind CSS · PostgreSQL (Neon) · Prisma ORM
Payments	Razorpay SDK, Test Mode (simulated settlement — see Limitations)
Deployment	Render (free tier Web Service), live at junior-pay.onrender.com
Total prompts exchanged with AI pair-programmer	250+ turns across a single continuous Gemini conversation used as a build log

2. Origin Story — From Idea to Product

The idea started from a simple, personal frustration: UPI works brilliantly for adults, but there was no clean way for a young child (roughly 5–10 years old, with no bank account, no phone, and no UPI ID) to pay for something without a parent physically handling the transaction. The original pitch, aimed at a senior software engineer at PhonePe, was described as "an update for a PhonePe-like company": a child scans a merchant QR, types in a parent's UPI ID, adds a note for what the payment is for, and the parent pays instantly from their own account — no calling, no manually forwarding a QR image.

2.1 Validating the Idea Against What Already Exists

Before building anything, the idea was checked against NPCI's **UPI Circle** (delegated payments), which by 2026 is already mainstream across BHIM, PhonePe, Google Pay, and Paytm. UPI Circle lets a primary user delegate spending to a secondary user with either full delegation (independent transacting within caps) or partial delegation (every transaction needs real-time approval — the mode built for minors).

Research surfaced a specific, defensible gap: UPI Circle still requires the secondary user to have their own linked mobile number / UPI ID and a smartphone with biometric setup in most implementations — realistic for teens, but not for a 5–10-year-old. Junior Pay's "scan → type parent's UPI ID → parent approves" flow needs no delegation setup and no child UPI ID at all, putting it in a lighter-weight, younger-skewing niche that UPI Circle does not serve well.

2.2 Naming and Repositioning

The product was deliberately repositioned from "a payment app for kids" (which would look like a naive competitor to an existing NPCI feature) to "a request-based, delegation-free layer for kids too young to even be added as a secondary user" — a narrower, credible niche. The name **Junior Pay** was chosen for being clean, memorable, and sounding like a genuine production FinTech product.

2.3 The Restart

After an initial prototyping phase, the project was deliberately restarted from a clean Next.js install (`npx create-next-app@latest junior-pay`) so that both the codebase and the AI-assisted learning context could reset around a disciplined, enterprise-style architecture: TypeScript, the App Router with a `src/` directory, Tailwind CSS, Prisma, and PostgreSQL, organized from Day 1 as a proper multi-tenant SaaS platform rather than a quick demo.

3. Problem Statement

Managing pocket money in the UPI era is broken. Cash allowances have largely disappeared, which leaves children out of the digital economy their parents transact in every day. At the same time, giving a child direct, unsupervised access to a standard UPI handle or app introduces real financial risk and parental anxiety, and most delegated-payment features on the market (like UPI Circle) still assume the child already has a phone and a linked identity of their own.

Junior Pay's thesis: children should be able to participate in digital payments through a **parent-monitored intermediary layer** — one that teaches financial literacy and responsible spending through a real-time approval loop and hard spending caps, instead of either excluding them entirely or handing them an unrestricted UPI identity.

4. How Junior Pay Works (Core Flow)

1. **The Request** — The Junior scans a merchant's UPI QR code (live camera scan) or manually enters the merchant's VPA, specifies an amount, selects a category/reason for the spend, adds a free-text note, and confirms with their 4-digit app PIN.
2. **The Security Gate** — The backend runs an idempotency check (blocks an identical duplicate request submitted within a 10-second window) and validates the request against the child's cumulative daily and monthly spending limits before it is allowed to reach a parent.
3. **The Guardian Notification** — The request instantly appears in the linked parent's dashboard (the "Parent Suite"), showing the merchant name/VPA, the amount, and the child's note.
4. **The Settlement** — On approval, the parent is taken through a Razorpay Test Mode checkout that simulates a real settlement. The backend verifies the payment (HMAC signature check) and flips the transaction to `APPROVED_AND_PAID`, updating both the Parent and Junior dashboards in real time. A parent can also decline a request outright.

Because both dashboards read directly from the same PostgreSQL source of truth via Prisma, there is no separate notification service needed to keep the two views in sync — the instant a transaction's status changes, the next page load (or refresh) on either side reflects it.

5. Feature List (Final Build)

Feature	What it does
Role-Based Authentication	Two distinct, secure entry portals: Juniors log in with a username + 4-digit PIN; Parents log in with a phone number + 6-digit PIN. Both roles get an encrypted, expiring session cookie.
QR & Manual VPA Entry	Kids can scan a real merchant UPI QR code with their device camera, or type a merchant's Virtual Payment Address directly if scanning isn't possible.
Real-Time Guardian Approval	Every payment request is pushed to the Parent Suite instantly, showing merchant details, amount, and the child's note, with one-tap Approve & Pay or Decline actions.
Multi-Guardian, Multi-Child Network	A single Junior can be linked to more than one parent (a "circle" of guardians), and a single parent can oversee more than one child — a true many-to-many relationship, not a single hard-coded pair.
6-Digit Pairing / Invite Codes	Parents generate a one-time, time-limited 6-digit code (expires in 15 minutes) that a child enters once to link the two accounts — no contact-list linkage or biometric setup required on the child's side.
Per-Connection Spend Limits	Each parent–child connection carries its own configurable Daily and Monthly limit type and cap, tracked cumulatively against the child's spend.
Limit-Exceeded Countdown Timers	When a child hits their configured limit, the UI shows a visual countdown to when the allowance window resets, instead of a bare error.
Anti-Spam Idempotency Protection	Identical pending transactions created within a 10-second sliding window are automatically blocked, preventing accidental double-submits or spam from a slow network.
Razorpay Test-Mode Checkout	A real Razorpay order-and-checkout flow (in sandbox mode) simulates the actual settlement experience for demo and portfolio purposes, including signature verification.
Transaction History / Passbook	Both Junior and Parent dashboards show a running history of requests with their status: Pending, Approved & Paid, or Declined.
Mobile-First Junior Experience	The child-facing app is designed as a centered, app-frame UI with a bottom tab bar (Hub/Pay, My Circle/Guardians, Passbook/History) to feel like a native mobile app.
Desktop-Style Parent Dashboard	The parent-facing side uses a sidebar navigation layout (Dashboard, Connect, History) suited to a management console rather than a mobile app.
Route-Level Security Middleware	Next.js middleware blocks direct URL access to /parent - dashboard or /junior - hub routes for unauthenticated or wrong-role sessions.

6. Screens & User Journeys

The application is split into three Next.js Route Groups — `(auth)`, `(parent)`, and `(child)` — which keep the codebase organized without affecting the public URLs, and let each side of the product have a completely different layout shell.

6.1 Junior Hub (Child Experience) — mobile app-frame shell

- **Hub / Pay** (`/junior-hub`) — the primary action center with quick actions to scan a QR or enter a VPA manually, plus a snapshot of recent activity.
- **My Circle / Guardians** (`/junior-hub/guardians`) — view every parent currently linked to the account and link a new one by entering a 6-digit pairing code.
- **Passbook / History** (`/junior-hub/history`) — a chronological list of past requests with their status (Pending / Paid / Declined).
- **Scanner** (`/junior-hub/scan`) — live camera QR scanner (with a manual fallback) that decodes the merchant's UPI URI and pre-fills the request form.

6.2 Parent Dashboard — sidebar console shell

- **Dashboard** (`/parent-dashboard`) — lists linked children, their daily limits, and any pending requests awaiting Approve & Pay / Decline.
- **Connect** (`/parent-dashboard/connect`) — generates a fresh 6-digit, 15-minute pairing code to onboard a new Junior, and lets a parent register/create their account with name and UPI ID.
- **History** (`/parent-dashboard/history`) — a full log of past approvals and declines across every linked child.

6.3 Public Landing Page

A marketing-style landing page (`/`) explains "How Junior Pay Works" to a first-time visitor, before they choose to log in as either a Parent or a Junior — used for the public demo, recruiter link, and LinkedIn launch.

7. Tech Stack & Architecture

Component	Technology / Tool	Purpose
Framework	Next.js 16 (App Router, src/dir)	Server-side rendering, file-system routing, and Server Actions used directly as the secure backend mutation layer (no separate REST-only backend).
Language	TypeScript	Type safety across frontend components, server actions, and the Prisma data layer.
Database	PostgreSQL, hosted on Neon	Relational, ACID-compliant storage — required for anything touching money and approval state, unlike a loosely-typed NoSQL store.
ORM	Prisma	Type-safe queries and schema migrations; a global singleton client pattern (globalThis.prisma) avoids connection exhaustion during Next.js hot-reload in development.
Styling	Tailwind CSS	Utility-first, responsive, mobile-first UI for the Junior app and a desktop-console feel for the Parent dashboard.
Payments	Razorpay SDK (Test Mode)	Generates sandbox orders, renders the checkout modal, and verifies payment signatures (HMAC SHA-256) server-side.
QR Scanning	Browser Barcode Detector API / jsQR-style canvas decoding	Hardware-accelerated, on-device decoding directly in the browser, including an inverted-color decode pass to reliably read dark PhonePe/Paytm-style QR codes without a native mobile app.
Auth & Sessions	Custom Role-Based Access Control (RBAC)	Encrypted session cookies distinguishing JUNIOR vs PARENT roles; Next.js middleware gates every protected route by role before a page or Server Action can run.
Deployment	Render (free-tier Web Service)	Builds and serves the Next.js app over HTTPS at a permanent .onrender.com URL, paired with an external uptime pinger to reduce cold-start latency.

7.1 Why This Stack

- **PostgreSQL over NoSQL:** anything touching money, allowances, or approvals needs strict, ACID-guaranteed consistency — if a child requests Rs. 50 and a parent approves it, every table must agree on that state.
- **Next.js Server Actions over a separate REST backend:** keeps the mutation logic co-located with the app while still running exclusively on the server, reducing surface area for a solo-built project.
- **Route Groups over flat routing:** lets the Junior and Parent experiences share one codebase and one database while getting entirely different navigation shells, without the URL structure leaking internal folder organization.

8. Database Schema (Prisma Models)

Model	Stores / Represents
ParentUser	Primary phone number, full name, secure 6-digit PIN, VPA identifiers, and the parent's generated invite codes.
Child	Unique login username, display name, and a 4-digit secure app PIN, plus their connections and transactions.
ChildConnection	The many-to-many relational bridge between a ParentUser and a Child — tracks limit type (DAILY / MONTHLY), the limit amount, and cumulative spend against it.
Transaction	Merchant VPA, merchant name, amount, approval status (PENDING, APPROVED_AND_PAID, DECLINED_BY_PARENT), and the child's note/category.
InviteCode	A secure, one-time, time-limited (15-minute) 6-digit onboarding code generated by a parent to link a new child account.

8.1 Design Evolution

The schema was deliberately refactored mid-project from a rigid one-parent-to-one-child structure into a proper many-to-many model. A dedicated `ChildConnection` join model was introduced specifically so that (a) a single child can be linked to multiple guardians, and (b) a single parent can manage multiple children — with per-connection limits rather than one global limit per child. This refactor was treated as its own phase ("The Database Refactor") before any UI built on top of it was touched.

9. Development Timeline & Iterations

Junior Pay went through two broad stages: an exploratory prototyping stage (where core ideas like QR scanning, pairing codes, and limits were first proven out) and a full restart into a structured 15-day, 5-phase build once the architecture, tech stack, and scope were locked in.

9.1 The Structured 15-Day Curriculum (Final Build)

Phase	Days	Focus
Phase 1	1–3	Unified authentication, middleware-based route security, and Route Group architecture.
Phase 2	4–6	RESTful/Server Action API design and application state management.
Phase 3	7–10	The Child Hub — QR/manual scanner, guardian network, transaction history.
Phase 4	11–14	The Parent Hub — approvals, invite/connect flow, history.
Phase 5	15	Final polish, landing page, and production deployment.

9.2 Documented Feature Iterations

Within that curriculum, several features went through explicit, multi-pass iteration rather than being built once and left alone:

- **Multi-tenant refactor (4 sub-phases):** database refactor → parent onboarding/registration UI → multi-pairing API → a security gate on the scan API confirming a typed UPI ID belongs to one of the child's actually-connected parents.
- **QR scanner (3 passes):** an initial simulated "pretend scan," then a real live-camera scan using a decoding library, then a further upgrade to the browser's native hardware-accelerated Barcode Detector API (plus an inverted-color decode pass) specifically to reliably read dark/inverted PhonePe- and Paytm-style QR codes.
- **Payment settlement (2 passes):** first a purely simulated "Approve" button with no real gateway, then a full Razorpay Test Mode integration — order creation, checkout modal, and HMAC signature verification.
- **Child UI (3 phases):** live camera polish → Parent UI enhancements (PIN modal, remove-junior action) → Child history and navigation.

10. Security & Engineering Edge Cases

Concern	How it's handled
Duplicate / spam requests	Idempotency window — the backend rejects an identical pending transaction created within the last 10 seconds, protecting against double-clicks or slow-network retries.
Database connection exhaustion	A global Prisma singleton (<code>globalThis.prisma</code>) is used so Next.js's development hot-reload doesn't spin up a fresh client — and a fresh connection pool — on every file save.
Unauthorized route access	Next.js middleware inspects the session role (JUNIOR vs PARENT) before allowing a request into <code>/parent-dashboard</code> or <code>/junior-hub</code> , redirecting unauthenticated or wrong-role sessions.
Cross-role data mutation	Server Actions explicitly re-validate the session role server-side before executing any database query or mutating a transaction's state — never trusting the client's UI state alone.
Untrusted / mismatched UPI IDs	The scan API checks that the sponsor UPI ID a child submits actually belongs to a parent currently in that child's authorized network, not just any string the child typed.
Payment integrity	Razorpay order IDs and payment IDs are verified server-side via an HMAC SHA-256 signature check before a transaction is ever marked as paid.
Pairing code abuse	Invite/pairing codes are single-use, numeric, and expire automatically after 15 minutes.

11. Notable Engineering Challenges Solved

Dark / inverted QR codes. Real PhonePe and Paytm QR codes are often dark-on-light or color-inverted relative to a typical QR, which broke a naive scanner. Fixed with a two-pass decode — a normal pass and an inverted-color (onlyInvert) pass — and later moved to the browser's native BarcodeDetector API for hardware-accelerated, low-end-device-friendly scanning.

Serverless database cold starts. Neon's free-tier Postgres spins down when idle, so early requests after inactivity threw connection-timeout errors (P1001). Solved with connection-string tuning (removing the pooler suffix where needed, adding connect_timeout) and, in production, an external uptime pinger to keep the underlying Render service warm.

Prisma + Next.js hot-reload connection exhaustion. Every hot-reload during development was instantiating a fresh Prisma Client, quickly exhausting the database's connection limit. Solved with the standard globalThis.prisma singleton caching pattern.

Payments to a real personal UPI handle in test mode. Using a real personal UPI ID as the 'merchant' during testing triggered protective blocking behavior from real UPI apps, since Razorpay Test Mode only pushes simulated funds through sandbox instruments (e.g. success@razorpay), not real money to real bank-linked handles — confirmed as expected sandbox behavior, not a bug.

Multi-tenant data modeling. The original one-parent-to-one-child schema couldn't express a child with multiple guardians or a parent with multiple children. Resolved by introducing a dedicated ChildConnection join model and re-writing the pairing and scan-authorization APIs against it.

12. Market Differentiation vs. Existing Solutions

The closest existing product is NPCI's **UPI Circle** (delegated payments), live across BHIM, PhonePe, Google Pay, and Paytm. Junior Pay was deliberately positioned into the gap UPI Circle leaves open for younger children, rather than as a head-on competitor.

UPI Circle (existing, NPCI-mandated)	Junior Pay (this project)
Rs. 5,000 per-transaction / Rs. 15,000 per-month hard cap, with a 24-hour cooling period after adding a secondary user	Parent-defined Daily/Monthly limits per connection, configurable per child
Each secondary user can accept delegation from only one primary user; one primary can create up to 5 circles	A child can be linked to multiple parents/guardians simultaneously (many-to-many)
Secondary user needs their own device with a passcode/fingerprint/face setup	No device, UPI app, or biometric setup required on the child's side at all
Primary user must add the secondary in advance via QR code or phone contacts	A child self-onboards instantly by entering a parent-issued 6-digit code — no pre-linking via contacts
Shows the merchant name only — no context on why money is being spent	Child attaches a free-text note and category reason to every request
Only available inside apps/devices that have specifically integrated the feature	Works in any modern mobile browser via a QR scan or manual VPA entry — no app install

Positioning: *Junior Pay is not pitched as a replacement for UPI Circle, but as a lighter-weight, delegation-free layer for the 5–10 age band that UPI Circle's device- and identity-dependent design doesn't reach.*

13. Limitations & Known Constraints

- **No real money movement.** Junior Pay does not move real funds. Direct API integration with consumer UPI apps (PhonePe, Google Pay, etc.) is restricted to licensed banking/PSP partners and is not accessible to an indie developer, so the settlement step uses Razorpay's **Test Mode**, which simulates the checkout and settlement experience with sandbox instruments only.
- **No 'Parent Suite' inside a real UPI app.** Because acquiring genuine API access to PhonePe/Google Pay's parent-side experience is not possible for this project, the "Parent Suite" is Junior Pay's own built dashboard rather than a real screen inside an existing payment app — the parent approves inside Junior Pay, then a real payment would need to happen through their own separately-installed UPI app.
- **Merchant-side notification is not automated.** A real merchant is notified automatically by their own UPI app when a genuine payment lands; since Junior Pay isn't moving real money, it does not attempt to fake a merchant-side notification and instead relies on the real UPI ecosystem for that step in an actual production deployment.
- **Free-tier hosting cold starts.** Render's free Web Service tier spins the server down after 15 minutes of inactivity, causing a 30–40 second delay on the first request after idle time. This is mitigated (not eliminated) with an external uptime pinger (e.g. cron-job.org) hitting the app every 10 minutes.
- **Serverless database cold starts.** The Neon PostgreSQL free tier can briefly go idle and needs to 'wake up,' which can surface as a connection-timeout error on the very first request in a while.
- **No native mobile app.** The Junior experience is a mobile-first web app styled to look and behave like a native app, not an installable iOS/Android application — QR scanning depends on browser camera and Barcode Detector API support rather than a dedicated on-device ML Kit integration.
- **Single-project, single-developer scope.** As a solo, portfolio-oriented build, the project does not yet include automated test coverage, formal accessibility auditing, or a production compliance review — appropriate for a demo/portfolio artifact, but not for handling real user funds.

14. Deployment

Junior Pay is deployed as a free-tier Web Service on **Render**, connected to a PostgreSQL database hosted on **Neon**. The live application is reachable at `junior-pay.onrender.com`.

14.1 Deployment Steps (as executed)

- Push the finalized codebase to GitHub (git add, commit, push).
- In the Render dashboard, create a New Web Service and connect it to the GitHub repository.
- Configure the build/start commands and environment variables (DATABASE_URL, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET) and choose the Free instance type.
- Deploy the Web Service — Render builds the Next.js app and serves it over HTTPS at a permanent .onrender.com URL.
- Register the live URL with a free uptime monitor (cron-job.org or UptimeRobot) pinging every 10 minutes, to reduce the impact of Render's 15-minute inactivity sleep and the resulting cold-start delay.

14.2 Free-Tier Characteristics

- **Permanent URL:** the .onrender.com link stays valid indefinitely as long as the service isn't deleted.
- **Sleep mode:** the app spins down after 15 minutes without traffic and cold-starts on the next request.
- **Monthly hours:** Render's free tier includes 750 hours/month — enough for one service to run continuously all month.

15. How to Use — Setup & User Guide

15.1 Running the Project Locally

- Clone the repository and install dependencies with `npm install`.
- Create a `.env` file with a PostgreSQL connection string (`DATABASE_URL`, e.g. from Neon) and Razorpay Test Mode keys (`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`).
- Run Prisma's migration/generate step against the schema to provision the `ParentUser`, `Child`, `ChildConnection`, `Transaction`, and `InviteCode` tables.
- Start the development server with `npm run dev` and open `localhost:3000`.

15.2 As a Parent

- Register/create a parent account with a name, phone number, UPI ID, and a 6-digit PIN.
- From the Dashboard's Connect page, generate a 6-digit pairing code (valid for 15 minutes) and share it with your child.
- Set a Daily or Monthly spend limit for that child on the connection.
- When a payment request appears, review the merchant, amount, and note, then Approve & Pay (via the Razorpay Test checkout) or Decline.
- Review past activity from the History page.

15.3 As a Junior (Child)

- Log in with your username and 4-digit PIN.
- From My Circle / Guardians, enter the 6-digit code your parent shared to link your account to theirs.
- From the Hub, scan a merchant's UPI QR code (or enter their VPA manually), enter the amount, choose a reason/category, add a note, and confirm with your PIN.
- Wait for your parent to approve — track the request's status from the Passbook / History tab.

16. Future Roadmap

Items explicitly identified during development as natural next steps, beyond the current portfolio build:

- A genuine bank/PSP partnership or licensed payment-aggregator integration to replace Razorpay Test Mode with real settlement, once regulatory and commercial requirements can be met.
- A native mobile app (or PWA install prompt) for the Junior experience, to use on-device vision APIs like Google ML Kit directly instead of the browser's Barcode Detector API.
- Push notifications (rather than dashboard polling/refresh) so parents are alerted the instant a request comes in.
- Expanded reporting for parents — spending trends per child, category breakdowns, and exportable statements.
- Formal automated test coverage and a security review, ahead of any real-funds pilot.

17. About the Author

Junior Pay was conceived, designed, and built end-to-end by **Shivam Nanda**, a B.Tech CSE student at Government College of Engineering, Gaya (Bihar Engineering University). He is the co-founder of **MindForge**, a peer learning community focused on skill-building and problem-solving, and hosts the YouTube podcast "**Learning Now by Shivam Nanda.**"

Link	Value
Live Application	junior-pay.onrender.com
LinkedIn	linkedin.com/in/shivam-nanda-472289418